PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

First Author, Second Author, Third Author
ITMO University, St. Petersburg, Russia
{author1, author2, author3}@itmo.ru

***Abstract*—Features for multi-table prediction tasks are built by aggregating neighbouring tables at a per-row seed time. Correctness requires every aggregate to respect that seed time along every join path and for every column, including columns written to a row after the row appears. We report violations of this requirement in three sources that we did not construct: a widely used feature-engineering library called without a cutoff-time table, as in its introductory example; the reference code written by the authors of this paper; and code generated by two language models under a neutral prompt (53% and 35% of programs that ran). The resulting AUC inflation ranges from 0.2 to 16 points and depends on the task more than on the defect. We demonstrate PITFALL, which runs a feature program twice, on the full database and on a database truncated at the seed time, and reports any divergence. The check does not parse the program, is language-agnostic, has no false positives by construction, names the offending columns in 0.2–7.6 s, and, by truncating one availability channel at a time, localises the leak to the table and column through which it enters. The univariate probe used by commercial AutoML platforms misses all three sources at DataRobot's default warning threshold, and H2O's more sensitive per-feature notification also fires on a correct pipeline.**

***Index Terms*—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code**

I. Introduction

In multi-table (relational) machine learning a prediction task is given as a table of triples *(entity, seed time, label)* over a database with foreign keys and timestamps [9]. Predictive features are obtained by walking join paths out of the entity and aggregating neighbouring rows, worth on the order of ten AUC points over the entity table alone [11]. Every aggregate may only read facts that existed at the seed time of the row it is computed for.

Leakage is a documented driver of irreproducibility in applied ML [1], [2], and this requirement is easy to violate for three reasons. Truncation is per row, so a harness that cuts the database once at the start of the test period says nothing about individual seed times. The requirement must hold along every join path, where the governing timestamp often belongs to a neighbouring table. Finally, a row can carry columns written later than the row itself, such as a review attached to an order; filtering by the order timestamp admits the row together with its future columns; this third case is the one most often missed.

We found violations in three independent sources, none constructed by us: `featuretools` [13] called without a cutoff-time table, the form of its introductory example [14]; the reference implementation written by the authors of this paper, found by our own checker; and single-shot generations from two language models under a prompt that does not mention leakage.

Existing safeguards do not cover this case. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker) use the same univariate probe: fit a model on each single feature and flag it when its score exceeds a threshold, e.g. Gini Norm 0.85 (AUC 0.925) for DataRobot and AUC 0.80 for H2O [7], [8]. Rule-based checkers such as `leakr` [6] enumerate a fixed list of patterns. Static analyzers for notebooks [3] and pipeline instrumentation such as `mlinspect` [4] target preprocessing order and train/test contamination, not temporal leakage. Asking a language model directly performs poorly: low-shot prompting reaches 0.19 precision at 0.52 recall, against $0.86/0.70$ for a trained classifier [5].

Contributions. (i) A simple execution-based check for feature programs, in the spirit of metamorphic testing: sound, one-sided, independent of the language and of any analysis of the code, extended with per-column availability and leak localisation by per-channel truncation. (ii) Evidence from three independent sources, on one public database, that point-in-time violations occur in ordinary practice. (iii) An execution-verified measurement of the rate at which two language models produce temporally incorrect feature code for one multi-table task, broken down by task construction and prompt guard strength. (iv) A working system and a four-scene demonstration in which the checker fires where the industrial probe is silent, stays silent on correct code, and localises the leak.

II. Point-in-Time Correctness

A. Definition

Let D be a database, t a seed time and φ a feature program. Each row r has a row timestamp $\mathrm{time}(r)$; a column c of r may in addition have its own availability timestamp $\mathrm{avail}_c(r) \geq \mathrm{time}(r)$. Write $D|_t$ for the database in which rows with $\mathrm{time}(r) > t$ are removed and values with $\mathrm{avail}_c(r) > t$ are replaced by NULL. Then φ is *point-in-time correct at* t iff

$$\varphi(D, t) = \varphi(D|_t, t). \tag{1}$$

For a cutoff shift $\delta \geq 0$ we write $I(\delta)$ for the AUC of a *fixed* learner A trained and evaluated on features computed with cutoff $t + \delta$, minus its AUC at $\delta = 0$. The learner is fixed because its choice is not a small effect: in one agent trajectory [12], swapping the boosting implementation on an identical feature set moved AUC by 11.5 points, comparable to the effects measured here.

B. Availability relation

Equation (1) generalises by replacing the time predicate with an *availability relation* $R(r, e, t)$, true when row r may be read when predicting for entity e at time t; co-emission and group leakage are other masks over the same machinery. Availability is defined per column: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days (maximum 147) after the order row it is attached to. Some columns have no availability timestamp at all, e.g. a mutable status field kept without history; for such a column the truncated database is identical to the full one and no execution-based check can decide it. We exclude one such column (`order_status`) from all conditions.

C. Why univariate probes miss it

If a leaked signal of fixed total strength is spread over d aggregate features, its visibility to any single-feature statistic decreases with d while its effect on a multivariate model does not; with enough aggregates a leak of any strength stays below a univariate threshold. Fig. 2 shows this on our data. When the leaked signal is concentrated in one feature the probe does detect it, as on one of our three tasks.

D. Differential execution and localisation

The check follows from (1): run φ twice and compare.

```
      full  = program(db, t, entities)
trunc = program(db.truncate(t), t, entities)
         verdict = (full != trunc)
```

A divergence is a proof of violation: the same program, given strictly less information, produced a different answer, whether through an aggregate, a join, or a statistic fitted on the full table. There are no false positives by construction. The guarantee is one-sided: a violation that does not manifest at the tested seed time is not observed. The program must be deterministic and re-runnable.

The same primitive localises the leak. The database has a finite set of *availability channels*: "rows of table T appearing after t" and "values of column c becoming known after t". Truncating one channel at a time and re-running φ identifies the channels on which the output changes and the output columns each of them affects. Masking exactly those channels and re-checking confirms that no other channel contributes; the mask is a diagnostic, not a fix, since any program passes on a pre-truncated input. This costs one run per channel (seven in our database).

Why check rather than always truncate? With one seed time per call, truncating the input is itself a fix. In the training tables of relational benchmarks each row has its own seed time, so truncation must happen per row inside the program, and that is what the check verifies. The check also applies to programs that read a live store rather than a data frame, and it reports a defect in code that will be reused, rather than masking it once.

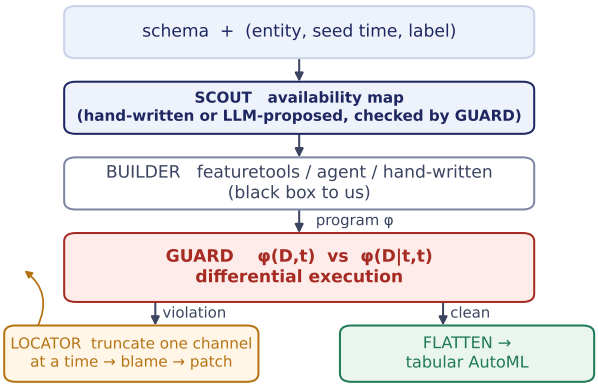

Fig. 1. PITFALL sits between any feature builder and any tabular AutoML. GUARD and LOCATOR are deterministic; the availability map consumed by SCOUT is hand-written in the current release and may be proposed by an LLM.

III. System

Fig. 1 shows the pipeline. SCOUT holds the availability map: which column governs the availability of which field, and which fields have none. In the current release the map is a small hand-written declaration (row-time column per table, value-time column per late-written field); an LLM may propose it from the schema. This is a schema question with an independently checkable answer, unlike "is there leakage here", which models answer poorly [5]. BUILDER is any producer of a feature program: a library, an agent, or a person. GUARD performs the differential check and returns a verdict, the list of diverging columns and the number of diverging cells. On a violation, LOCATOR runs the per-channel truncation and reports which channel feeds which output column. Clean programs are flattened into a single table and handed to an unmodified tabular AutoML system such as AutoGluon [15]. The core is about 170 lines of Python over pandas and numpy, with no dependency on the builder's language.

Validation of the checker. On six reference programs with known verdicts (correct; leak in the entity's own history; leak only through a join; both; no cutoff; and our original "correct" code, see below) at three seed times, the checker was right on 18 of 18 combinations. As a negative control we set the seed time to the maximum timestamp in the database, where every aggregation is correct by construction and zero alerts are expected. The first run of this control produced alerts: two null outputs were compared as unequal, so programs that returned nothing were reported as violations. The control ran before the main experiment, as pre-registered; a re-audit of the 23 verdicts issued until then found one false.

TABLE I

SOURCES OF POINT-IN-TIME VIOLATION. ROWS 1, 2 AND 5 WERE NOT CONSTRUCTED BY US; ROW 4 IS A CONTROLLED REGIME. "PROBE" IS THE MAXIMUM SINGLE-FEATURE AUC ON THE LEAKING FEATURES; DATAROBOT WARNS AT GINI NORM 0.85 (AUC 0.925), H2O DRIVERLESS AI NOTIFIES AT AUC 0.80.

Source	Inflation, pp	Probe	DataRobot / H2O
`featuretools`, no cutoff	$+14.8 \ldots +16.3$	0.76–0.83	silent / notifies 2/3
Our reference code, task B	$+3.1 \ldots +5.3$	0.62–0.69	silent / silent
same defect, tasks A and C	$-0.1 \ldots +1.0$	0.71–0.86	silent[†]
Leak through a join path only	$+3.0 \ldots +5.1$	unchanged	silent / silent
LLM code, neutral prompt	median $+0.1$	53% / 35%	of programs violate

[†]On task A the H2O notification fires on the *correct* pipeline too (Section IV-C).

IV. EVIDENCE

Setup. Olist [16], a public Brazilian marketplace database: 7 tables, 112,650 order line items, September 2016 to October 2018. Three tasks in RelBench [9], [10] form (seller activity, A; seller review quality, B; product demand, C), three seed times (January, April, July 2018), 90-day horizon, training on all earlier seed times; 750–870 sellers and 4,500–5,800 products per seed time. One LightGBM configuration with a fixed seed everywhere; two CPU cores.

A. The cost of a shifted cutoff

$I(\delta)$ increases monotonically over $\delta \in [0, 90]$ days on tasks A and B at all three seed times: a one-month error costs 5.8–11.5 points, and removing the cutoff entirely yields $+12.3$ to $+27.0$ (A) and $+5.8$ to $+18.0$ (B). On task C we control the cutoff separately for the entity's own history and for aggregates reached through a join to the seller: leaking only through the join costs $+3.0$ to $+5.1$ points and is not visible to the probe (Section IV-C).

B. Sources of violation

Table I summarises the sources.

A library, called as in its introductory example. `featuretools` 1.31.0 with `ft.dfs` and no cutoff-time table inflates AUC by 14.8, 16.3 and 15.6 points on the three seed times of task C. The library supports cutoff times but does not indicate that they are needed.

Our own reference code. Our reference implementation filtered history by order time and took all columns of the admitted rows. Two of those columns have their own, later timestamps: the review score and the delivery outcome. Across the three seed times, 5.0%, 5.0% and 2.0% of orders placed before the seed time carry a review written after it. The checker flagged four columns (`review_score_mean`, `review_score_min`, `late_mean`, `delay_days_mean`) and no others. The cost is $+3.09$, $+5.26$ and $+3.78$ points on task B, but only $+0.16$ to $+0.31$ on A and -0.10 to $+0.98$ on C.

Published expert SQL. In the hand-written feature SQL of the RelBench user study [9] (15 files, 172 joins), a manual audit found two violations, about 1% of joins; a syntactic scanner raised eight candidates, six of them false. This audit was static (the database was not reachable from our environment).

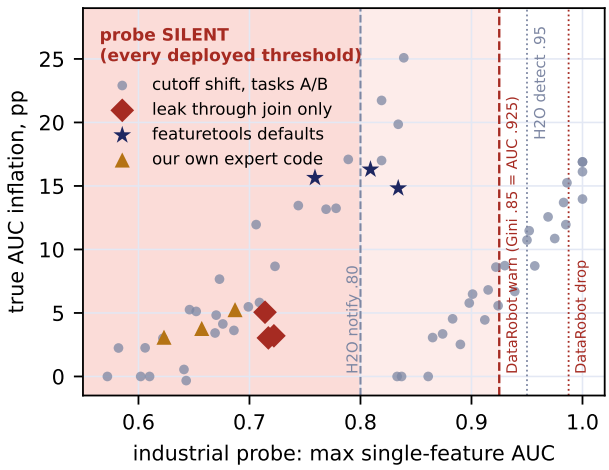

Fig. 2. Probe value against true inflation. Points left of 0.925 are violations the DataRobot probe does not report, including all three sources of Table I; the H2O notification at 0.80 catches two of the three `featuretools` points and none of the others.

C. What the industrial probe reports

Fig. 2 plots the probe value against true inflation. The clearest case is leakage through a join: the probe returns $0.714/0.722/0.717$ under a correct cutoff and the same $0.714/0.722/0.717$ under leakage, while the model gains 3.0–5.1 points. On task B no condition, including the complete absence of a cutoff ($+18.0$ points), reaches the DataRobot threshold, and only shifts of two months or more reach H2O's 0.80. The more sensitive threshold also errs in the other direction: on task A the correct pipeline scores $0.837/0.861/0.833$, above 0.80 at every seed time, because recency is a legitimate and strong predictor of churn. Lowering the threshold trades misses for false alarms; on these tasks no threshold separates the two.

D. Generated feature code

We asked two models, `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code`, for a Python function computing features for task C, giving the schema and the training table with the seed-time column named, and nothing else; the words "leakage" and "point-in-time" do not appear. Each generation is single-shot with a clean context at temperature 0.7; verdicts come from the checker; five of the flagged programs were additionally read by hand and confirmed. Of 40 generations per model, 30 and 17 ran (the rest failed to execute and are excluded, which may bias the rate in either direction). Of those, 53.3% [36.1, 69.8] and 35.3% [17.3, 58.7] were incorrect. The failure mode is the same in every case examined: the model filters the entity's own history by the primary event timestamp and does not account for the later timestamp of the joined entity. We make no claim about model capability in general.

Task construction is decisive: on a second task with a single seed time per entity, only the entity's own history and no

Fig. 3. Scene 4 of the demonstration page: LOCATOR names the availability channel behind each diverging column.

secondary time axis, both models produce 0 violations in 24 and 28 running programs. Prompt guards help partially (30–40 generations per cell, 9–30 running): moving from a neutral prompt to a general reminder to a per-aggregate requirement reduces the pooled rate from 46.8% [33, 61] to 41.2% [26, 58] to 14.6% [7, 28], but not to zero.

E. Cost of a violation is task-dependent

The same defect (the same secondary time axis and the same code) costs 0.16–0.31 points on task A, -0.10–0.98 on C and 3.09–5.26 on B; among model-generated programs the median inflation is 0.11 points (range -2.2 to $+2.0$). The cost depends on how many features are affected and how much the target depends on them, and neither is known from the fact of the violation alone. Correctness therefore has to be tested separately from the quality metric, which an execution-based checker does and a metric-based heuristic cannot.

V. DEMONSTRATION

The demonstration runs offline on a laptop in about two minutes (`python3 demo.py`), prints per scene a verdict, the diverging columns, the probe value at both thresholds and the true inflation, and renders a self-contained page (Fig. 3). Attendees can edit a feature function (`-program`) and re-run the check.

Scene 1 — library defaults. `ft.dfs` without a cutoff-time table. Verdict VIOLATION in 7 s, 11 diverging columns, 19,334 cells; probe 0.809 (silent for DataRobot, H2O notification); inflation $+16.3$ points. **Scene 2 — our own code.** Verdict VIOLATION in 0.2 s, naming the four review and delivery aggregates; probe 0.687, silent at both thresholds; $+5.3$ points. **Scene 3 — the fix.** The same program with the availability relation fixed and nothing else changed. Verdict CLEAN, outputs identical, inflation 0.00 at all three seed times; the negative control for the first two scenes.

Scene 4 — localisation. LOCATOR truncates the seven availability channels one at a time. For Scene 2 it attributes the two review aggregates to `review_score` becoming known after t and the two delivery aggregates to the delivery date; for Scene 1 it attributes all eleven columns to `items` rows appearing after t. Masking those channels alone re-checks CLEAN, so no other channel is involved.

VI. LIMITATIONS

(i) One-sided. The check proves violation, not correctness, and only with respect to the observed database and the chosen mask. *(ii) Undecidable columns.* Mutable fields kept without history have no availability timestamp and cannot be checked by execution. *(iii) Dependence on the availability map.* A wrong timestamp in the map means the wrong thing is checked. *(iv) Scope.* One database, three tasks, three seed times; two models at one price tier under specific prompts. *(v) Final-program metric undercounts.* An agent's validation-driven selection can discard a leaking intermediate trial (observed once); checking only the final artifact understates the rate.

VII. CONCLUSION

Point-in-time violations in multi-table feature pipelines occur in library defaults, expert code and generated code, and the univariate probe used to catch them misses the regimes reported here. Differential execution decides the question in seconds without analysing the program and localises the leak with the same primitive.

DATA, ETHICS AND AVAILABILITY

All experiments use the public Olist dataset (Kaggle, CC BY-NC-SA 4.0), anonymised by its publisher; no human subjects were involved. Language-model outputs are released unedited. Code, data and all reported numbers: https://github.com/stas1f1/pitfall.

REFERENCES

[1] S. Kaufman, S. Rosset, and C. Perlich, "Leakage in data mining: formulation, detection, and avoidance," in *Proc. KDD*, 2011.
[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.
[3] C. Yang *et al.*, "Data leakage in notebooks: static detection and better processes," in *Proc. ASE*, 2022.
[4] S. Grafberger, S. Guha, J. Stoyanovich, and S. Schelter, "mlinspect: a data distribution debugger for machine learning pipelines," in *Proc. SIGMOD*, 2021.
[5] N. Alturayeif and J. Hassine, "Data leakage detection in machine learning code: transfer learning, active learning, or low-shot prompting?" *PeerJ Computer Science*, 11:e2730, 2025.
[6] C. Isabella, "leakr: data leakage detection," CRAN package, 2025.
[7] DataRobot documentation, "Data quality assessment: target leakage," docs.datarobot.com/en/docs/reference/data-ref/data-quality-ref.html, 2026.
[8] H2O.ai, Driverless AI 1.11 User Guide, "Data leakage and shift detection," docs.h2o.ai, 2026.
[9] J. Robinson *et al.*, "RelBench: a benchmark for deep learning on relational databases," in *NeurIPS Datasets and Benchmarks*, 2024.
[10] J. Gu, R. Ranjan, C. Kanatsoulis *et al.*, "RelBench v2: a large-scale benchmark and relational data repository," in *ICLR Workshop on Foundation Models for Tabular and Structured Data (DATA-FM)*, 2026.
[11] M. Wang *et al.*, "4DBInfer: a 4D benchmarking toolbox for graph-centric predictive modeling on relational DBs," in *NeurIPS Datasets and Benchmarks*, 2024.
[12] "RelAgent: agentic feature engineering for relational data," arXiv preprint arXiv:2605.07840, 2026. (Preprint; not peer reviewed.)
[13] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: towards automating data science endeavors," in *Proc. DSAA*, 2015.
[14] Featuretools 1.31 documentation, "Handling time," featuretools.alteryx.com, 2026.
[15] N. Erickson *et al.*, "AutoGluon-Tabular: robust and accurate AutoML for structured data," arXiv preprint arXiv:2003.06505, 2020.
[16] Olist, "Brazilian e-commerce public dataset," Kaggle, 2018.